Introduction to the Strategy of Boys and Girls Kaggle Competition

PO-LIAN LIAO
114423053
williamliao2013@gmail.com

CHENG-WEI LIAO
114423068
apple84621@gmail.com

MING-JHE HO
114423069
victort509dm@gmail.com

XIN-HAN CHEN
114423014
hess0623977@gmail.com

ABSTRACT

This study aims to build a gender prediction model that can easily adapt to new data, it helps solve the unbalanced data and complex non-linear problems in the "Boys and Girls" dataset.

During the data preprocessing stage, we cleaned up messy category labels and removed extreme values based on normal human physical limits. We also capped long-tail features at the 99th percentile to prevent outliers. For feature transformation, we used the BGE-Large model to extract deep meanings from text and used PCA to reduce the data size. At the same time, we strictly kept Target Encoding and Random Forest Iterative Imputation inside a 10-Fold cross-validation process, it ensured absolutely "zero data leakage." After that, we used a multi-voting Permutation Importance method to select the best features and scientifically remove useless noise.

The final model architecture perfectly combined Stacking and Blending techniques. We used XGBoost, LightGBM, and Random Forest as our base models, and handed the final prediction over to Logistic Regression. To lower the risk of overfitting, our system further used three different parameter settings: "conservative, moderate, and aggressive" to average out the final probability scores. Following this strict experimental pipeline, our team proudly achieved an excellent cross-validation accuracy of 0.8792, and scored 0.8826 on the competition's Public Leaderboard, with a final score of 0.8638 on the Private Leaderboard.

1. DATA PREPROCESS

1.1 Introduction

This chapter documents the preprocessing pipeline from raw data to the clean feature matrix used for model training.

The "Boy or Girl" dataset contains 423 training and 426 test samples, each with 9 features across three modalities: numerical (height, weight, sleepiness, iq, fb_friends, yt), categorical (star_sign, phone_os), and unstructured text (self_intro). The target variable gender is imbalanced at approximately 3:1.

Three preprocessing challenges shape our design: (1) severe data quality issues — Excel error codes in yt, physically impossible values in height/weight, and billion-scale outliers in fb_friends; (2) 16%–25% missing rates across all features, potentially following MNAR patterns where missingness itself carries predictive signal; (3) data leakage risk — with only 423 samples, globally fitting any distribution-dependent transformation risks inflating cross-validation estimates.

Accordingly, our pipeline adheres to three principles: evidence-based cleaning thresholds grounded in physical plausibility, multi-layered imputation that preserves missingness as a learnable feature, and strict encapsulation of all label-dependent transformations within each cross-validation fold to ensure zero data leakage. The following sections cover: Section 1.2 exploratory analysis, Section 1.3 data cleaning, Section 1.4 missing value imputation. Feature engineering and leakage-free pipeline design are presented in Chapter 2.

1.2 Exploratory Data Analysis

Prior to any transformation, we conducted exploratory analysis to establish the empirical basis for subsequent decisions.

1.2.1 Missing Rate and Data Quality Overview

All feature columns exhibit missing rates between 16% and 25% in both training and test sets (Figure 1), indicating a structurally consistent pattern rather than isolated anomalies. Beyond genuine missingness, corrupted entries include Excel error codes (#NUM!) in yt, negative values and magnitudes exceeding 10^{111} in height/weight, and billion-scale outliers in fb_friends.

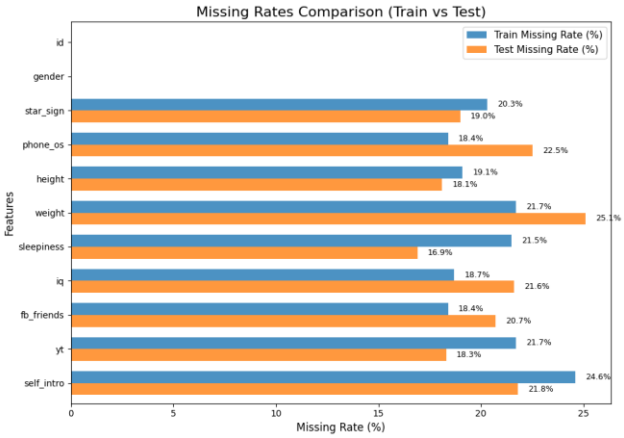

Figure 1. Missing rate comparison across features between training and test sets.

1.2.2 Gender-Discriminative Feature Analysis

As shown in Figures 2 and 3, height and weight exhibit the most pronounced between-group separation with limited distributional overlap, establishing them as the primary classification drivers. The remaining numerical features show considerably smaller between-group differences. For the text feature self_intro, word frequency analysis (Figure 4) reveals clear gendered language patterns —

gender=1 favors terms like "handsome" and "hard," while gender=2 favors "beautiful" and "cute" — confirming exploitable classification signal and motivating the semantic embedding in Chapter 2.

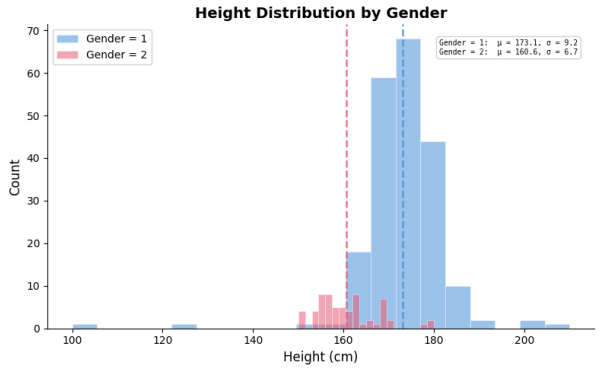

Figure 2. Height distribution by gender, with dashed lines indicating group means.

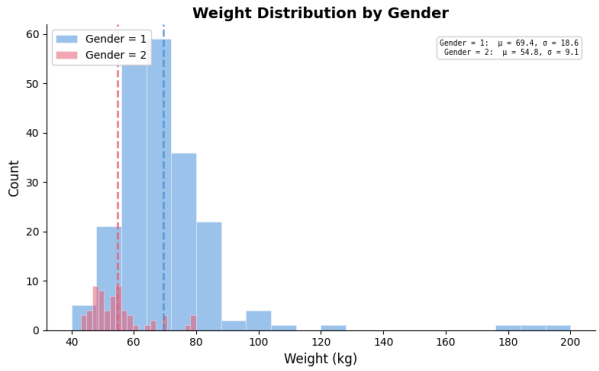

Figure 3. Weight distribution by gender, with dashed lines indicating group means.

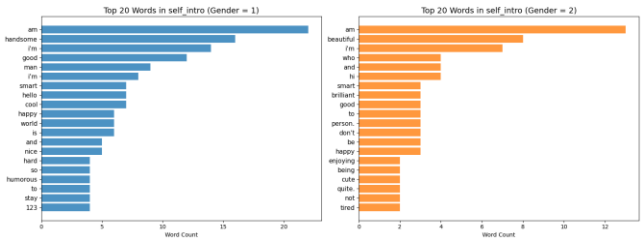

Figure 4. Top-20 word frequencies in self_intro by gender.

1.2.3 Mutual Information Quantification

We quantified each feature's predictive information using Mutual Information (Figure 5), with a temporary median/mode imputation applied solely to satisfy MI's complete-data requirement. Height and weight dominate with MI values an order of magnitude above the rest; self_intro ranks third, quantitatively validating the word frequency observations. Star_sign, sleepiness, and phone_os register zero MI, though MI captures only pairwise association and cannot detect value emerging through feature interactions or encoding transformations. Their final retention is deferred to Chapter 2.

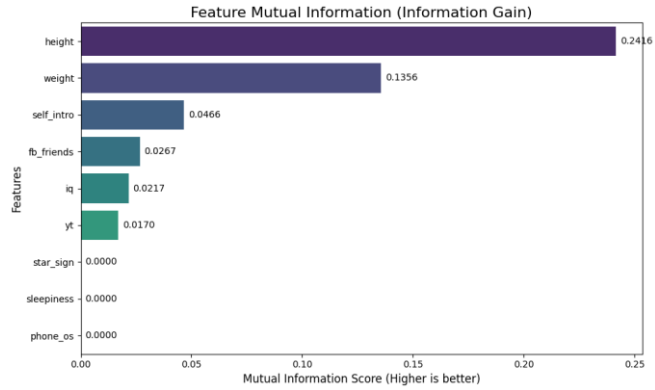

Figure 5. Mutual Information score of each feature with respect to the target variable.

1.3 Data Cleaning

The cleaning strategy addresses two aspects: outlier filtering for numerical features and label normalization for categorical features.

1.3.1 Physical Boundary Filtering

The outliers in this dataset do not arise from natural distributional variation but from human input errors (e.g., negative heights) and system-level artifacts (e.g., Excel overflow producing values on the order of 10^{111}) — magnitudes far beyond any statistically derived threshold. We therefore opted against IQR or Z-score methods and adopted domain-knowledge-based physical boundary filtering, converting values outside plausible ranges to NaN for downstream imputation. The filtering bounds for each feature are summarized in Table 1.

Table 1. Physical boundary filtering thresholds for each numerical feature.

Feature	Min Bound	Max Bound	Rationale
height	100cm	250cm	Exclude negative values and extreme outliers (e.g., 1e+111)
weight	30 kg	200kg	Exclude negative values and extreme outliers
iq	50	200	Reasonable human IQ range
sleepiness	0 hr	24hr	Maximum 24 hours in a day
fb_friends	0	5000	Reasonable range
yt	0	Dynamic 99th Percentile	Exclude Excel errors (#NUM!) and negative values

1.3.2 Phone_os Label Normalization

The raw phone_os column contains inconsistent formats such as "apple," "Apple," "iOS," and "ANDROID." We normalized these into three standard categories via keyword matching — "Apple," "Android," and "Other" — preventing identical operating systems from being treated as distinct categories.

1.4 Missing Value Imputation

1.4.1 Missing Indicators

Prior to any imputation, we created binary missing indicators (_is_missing) for weight, height, and yt. If missingness follows an MNAR pattern (e.g., a particular gender being more inclined to skip the weight field), the missingness itself becomes a predictive signal; additionally, indicators allow the model to distinguish genuinely observed values from imputed estimates. Indicators are created only for these three core numerical features with non-zero MI scores.

1.4.2 Imputation Method Selection

We adopted IterativeImputer with RandomForestRegressor. To validate this choice, we conducted a controlled ablation study varying only the imputation method while holding all other pipeline components identical across four strategies (Table 2). ROC-AUC scores range from 0.9227 to 0.9319 — a spread of 0.009, well within each method's per-fold standard deviation of 0.06–0.07 — indicating no statistically significant difference.

We nevertheless select IterativeImputer on the grounds of qualitative imputation plausibility. Median imputation fills each column with a single identical value, destroying inter-feature covariance — all imputed samples collapse to the same BMI and other derived features. IterativeImputer instead treats each missing column as a target variable, training a RandomForest on the remaining columns to produce individualized estimates that preserve nonlinear covariance structure. To prevent data leakage, the imputer is strictly encapsulated within each CV fold; the full architecture is detailed in Section 2.2.

Table 2. Ablation study: downstream performance across four imputation strategies under identical pipeline conditions

Imputation Strategy	ROC-AUC (10-Fold)
Simple Imputer (Median)	0.9319 (±0.0607)
KNN Imputer (k=5)	0.9317 (±0.0639)
LightGBM Imputer	0.9308 (±0.0702)
Random Forest Imputer	0.9227 (±0.0727)

2. FEATURE ENGINEERING AND SELECTION

2.1 Feature Engineering

After basic data cleaning, we constructed informative features to improve model performance and stability, specifically addressing the challenges of a high-dimensional feature space on a small dataset.

2.1.1 Derived Features

To capture body shape dynamics more effectively than using height and weight independently, we introduced two complementary derived features: BMI (capturing non-linear interactions) and height-weight ratio (capturing linear proportions). Additionally, we applied a log1p transformation to fb_friends and yt to compress their heavily right-skewed distributions, mitigating the influence of extreme outliers and stabilizing tree-based learning.

2.1.2 NLP Semantic Features

self_intro was the only free-text field in the dataset and appeared to contain useful gender-related signals in wording, tone, and writing style. To capture this information, we used BAAI/bge-large-en-v1.5 instead of TF-IDF, since embedding models can represent contextual meaning rather than simple word frequency. In preliminary experiments, BGE-Large outperformed the lighter MiniLM-384 model in cross-validation, so we selected it as the final text encoder.

Each self-introduction was converted into a 1024-dimensional embedding vector. However, this was far larger than the original tabular feature space, which contained only around a dozen structured variables. If used directly, the text embeddings could dominate the overall feature space and weaken the contribution of the original features, while also increasing the risk of overfitting in a small dataset. We therefore applied PCA to reduce the embeddings to 5 principal components, preserving the main semantic information while keeping the text features at a scale more balanced with the structured inputs.

2.1.3 Target Encoding

For the non-ordinal categorical variables star_sign and phone_os, we used target encoding instead of one-hot encoding. In a small dataset, target encoding is often more practical because it avoids unnecessary dimensional expansion while still preserving predictive information.

Since rare categories can easily lead to overfitting, we used a smoothing parameter of 20. This regularization pulls category-specific estimates toward the global mean, making the encoded values more stable and less sensitive to small sample counts.

2.2 Leakage Control

2.2.1 Leakage Risks

Data leakage was a key concern in this project. Any transformation fitted on the full dataset before cross-validation can allow information from the validation fold to influence the training process and lead to overly optimistic results. This risk applied to all preprocessing steps that learn parameters from data, including target encoding, PCA, and the Random-Forest-based IterativeImputer.

2.2.2 Solution of Leakage

To prevent leakage, we implemented a unified preprocessing function, preprocess_fold, and ran it separately within each fold. All data-dependent steps were fitted only on the training split and then applied to the validation split. This included target encoding, PCA on the text embeddings, and the Random-Forest-based IterativeImputer, followed by the creation of derived features such as BMI, height-weight ratio, and log-transformed variables. As a result, the validation fold remained unseen throughout preprocessing, making the cross-validation scores a more reliable estimate of generalization performance.

2.3 Feature Selection

2.3.1 Why Permutation Importance

Because the dataset was small (423 samples) and included text-derived features, feature selection was necessary to reduce the risk of treating noise as signal. We initially examined LightGBM's built-in split importance, but this metric can overestimate the importance of continuous or high-cardinality features. We therefore used permutation importance, which evaluates feature contribution by measuring the drop in validation performance after shuffling each feature. ROC-AUC was used as the evaluation metric because the dataset was imbalanced and AUC provides a more reliable measure than accuracy in this setting.

2.3.2 Multiple Voting System

Permutation importance can vary across random splits, especially in small datasets. To improve stability, we evaluated feature importance using 5 random seeds, each with 5-fold cross-validation.

For each seed, permutation importance was computed in every fold and then averaged across the five folds. A feature was retained only if its averaged importance was positive in at least 3 of the 5 seeds. This voting scheme favored features with stable contributions across repeated evaluations and reduced the chance of retaining noisy variables.

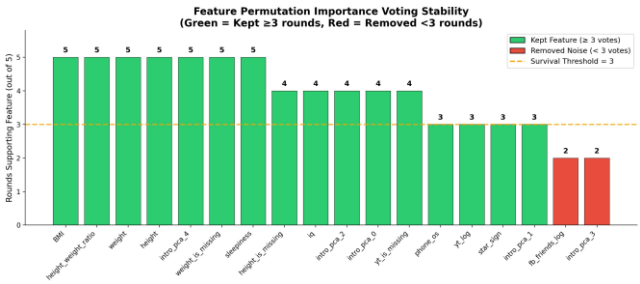

Figure 6. Permutation Importance with Voting Support.

2.4 Final Feature Set

After feature engineering and the voting-based selection procedure, 2 features were removed (intro_pca_3 and fb_friends_log) and 16 features were retained for the ensemble stage.

The final retained feature set consisted of the following:

- Original and derived features: height, weight, BMI, height_weight_ratio, iq, sleepiness, yt_log
- Missing-value indicator features: weight_is_missing, height_is_missing, yt_is_missing
- Categorical encoded features: star_sign, phone_os
- Text semantic features: intro_pca_0, intro_pca_1, intro_pca_2, intro_pca_4

3. EXPERIMENTAL PROCESS

This study aims to build a stable prediction model that can generalize well to new data. To find useful connections among many features while avoiding the risks of overfitting and data leakage, we designed a Stacking classification model. This model is based on three base models and includes a feature selection step. Please see Figure 7 for the model flowchart, and the detailed steps are explained in the following sections.

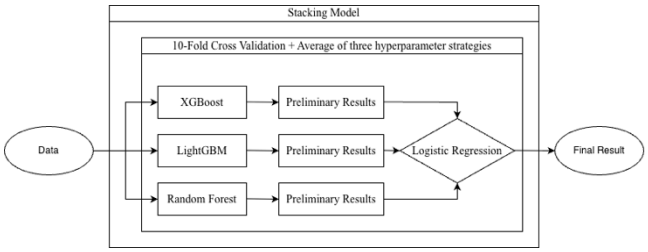

Figure 7. Experimental flow chart.

3.1 Base Classification Model Selection

Our stacking model uses three different tree-based classification models as its base models(Level 0). To handle the imbalanced data issue (since there is a clear difference in the number of males and females in the "Boys and Girls" dataset), we added a class weight balancing mechanism. The three models we chose are XGBoost (Extreme Gradient Boosting), LightGBM (Light Gradient Boosting Machine), and Random Forest.

3.1.1 Extreme Gradient Boosting

XGBoost is a powerful and efficient gradient boosting decision tree algorithm. It is very good at finding complex, non-linear relationships in data. In this study, to match our overall strategy for handling imbalanced data, the model dynamically calculates the ratio of positive to negative samples (scale_pos_weight) to adjust its learning bias. This ensures that the features of the minority class are also fully learned.

3.1.2 Light Gradient Boosting Machine

Compared to traditional decision trees that grow level by level (Level-wise), LightGBM uses a strategy that grows by leaves (Leaf-wise). This gives it a clear advantage in reducing errors and discovering tiny feature details. To prevent this growing strategy from overfitting on complex data, we strictly controlled the number of leaves and the depth of the tree. We also turned on the class weight balancing mechanism (class_weight='balanced') to automatically penalize the model for prediction errors on the majority class.

3.1.3 Random Forest

Random Forest is an ensemble algorithm based on the Bagging concept. It makes a majority vote by combining many independent decision trees that are trained on random feature samples. In our Level 0 model group, Random Forest plays a key defensive role. Compared to the gradient boosting models (XGBoost and LightGBM) that aggressively try to fit errors, Random Forest effectively reduces the overall prediction variance. This provides a stable balancing force for the stacking model.

3.2 Final Predictor Model Selection

In the final stage (Level 1) of our Stacking architecture, we use Logistic Regression as the final predictor. Its main goal is not to learn patterns directly from the original data. Instead, it uses the "predicted probabilities" produced by the three base models in the first layer (Level 0) as its input features.

By learning how XGBoost, LightGBM, and Random Forest perform and where they tend to make mistakes on different samples, Logistic Regression can automatically calculate the best decision weights. This allows it to combine the opinions of all three models and output a final prediction that is both highly optimized and very stable.

3.3 Model Training and Validation

3.3.1 Training Data and Test Data Segmentation

Instead of employing a traditional hold-out validation (e.g., an 80/20 train-test split), this study utilizes k-fold cross-validation to address the limitations of our relatively small dataset, where a single split might yield biased evaluation results. To ensure the stability and reliability of the model assessment, the following configurations were implemented:

- Stratified Sampling：To mitigate potential bias caused by data imbalance, stratified sampling was applied to maintain a consistent distribution of categories (Male/Female) across both training and testing sets.
- 10-Fold validation：The labeled dataset was partitioned into 10 folds. In each iteration, nine folds served as the training set while the remaining fold functioned as the validation set. The final performance metrics represent the average of all 10 iterations. Compared to 5-fold validation, this approach provides a more stable estimation of model performance, particularly for smaller datasets.

- Reproducibility：We set a random seed of 42 to ensure the splits are reproducible, allowing for a consistent and fair comparison across all experimental runs.

3.3.2 Training Procedure

This study employs a diverse suite of machine learning models, primarily centered around Gradient Boosting Decision Trees (GBDT)—specifically XGBoost and LightGBM—with Random Forest serving as the baseline for comparative analysis. These models were selected for their strong ability to capture non-linear relationships in structured data.

To ensure experimental rigor and prevent data leakage, a "Zero-Leakage" preprocessing pipeline was implemented. All data-dependent transformations—such as Target Encoding (with smoothing = 20), Iterative Imputation (Random Forest, n_estimators = 50, max_iter = 10), and PCA—are performed within each cross-validation fold, ensuring unbiased evaluation.

Regarding feature engineering for textual data (self_intro), the BGE-Large (BAAI/bge-large-en-v1.5) semantic embedding model was utilized for transformation, with missing values imputed as "no description." To mitigate the curse of dimensionality while preserving core semantic information, PCA was subsequently employed to reduce the high-dimensional vectors into five dimensions (intro_pca_0 to intro_pca_4).

Finally, a dual-layer ensemble approach is used. Stacking combines XGBoost, LightGBM, and Random Forest with Logistic Regression (class_weight='balanced') as the final learner to aggregate the predictions. In the final inference stage, blending further integrates predictions from models with different hyperparameter settings to improve overall stability and robustness.

3.3.3 Hyperparameter Settings

Hyperparameter configuration has a significant impact on model performance. In this study, we evaluated the effectiveness of automated search methods (e.g., Optuna) against manually predefined settings. Given the risk of overfitting to the validation set—a common challenge with smaller datasets—we ultimately opted for three distinct manual configurations. These settings were designed to represent different bias-variance tradeoff:

- Conservative：Characterized by shallower trees, fewer estimators, and a lower learning rate. This configuration prioritizes the reduction of model complexity to mitigate overfitting risks.
- Moderate：Features intermediate tree depth, a standard number of training rounds, and a balanced learning rate, aiming for an optimal equilibrium between bias and variance.
- Aggressive：Employs deeper trees and an increased number of estimators coupled with a lower learning rate, allowing for more iterations to ensure full convergence. This setting focuses on maximizing the model's fitting capacity.

The actual hyperparameter settings for each model are shown in the table below:

Table 3. Hyperparameter Configurations for XGBoost

Parameter	Conservative	Moderate	Aggressive
max_depth	3	5	7
learning rate	0.05	0.03	0.01
n_estimators	150	200	300
reg_lambda	5.0	1.0	0.1

Table 4. Hyperparameter Configurations for LightGBM

Parameter	Conservative	Moderate	Aggressive
max_depth	3	5	7
num_leaves	15	31	63
learning rate	0.05	0.03	0.01
n_estimators	150	200	300

Table 5. Hyperparameter Configurations for Random Forest

Parameter	Conservative	Moderate	Aggressive
max_depth	4	6	8
n_estimators	150	200	300
min_samples_leaf	4	2	1

Furthermore, to address the class imbalance inherent in the dataset, XGBoost was configured with the scale_pos_weight parameter, while LightGBM and Random Forest employed the class_weight='balanced' setting. To ensure the reproducibility of all experimental results, a fixed random_state=42 was applied across all models.

3.3.4 Evaluation Metrics

To comprehensively evaluate the model's performance, this study employs multiple evaluation metrics. The specific definitions and significance of each metric are detailed below.

3.3.4.1 Accuracy

The proportion of total correct predictions among the entire set of examined cases.

3.3.4.2 F1-score

The harmonic means of Precision and Recall, which provides a balanced evaluation metric especially suitable for scenarios involving class imbalance.

3.3.4.3 Area Under the ROC Curve (AUC)

Measures the model's discriminative ability to distinguish between different categories. As it is particularly sensitive to performance in class-imbalanced scenarios, it is designated as a primary evaluation metric for this study.

3.3.4.4 Log Loss

By measuring the divergence between predicted probabilities and ground-truth labels, Log Loss reflects the model's probabilistic calibration capability, ensuring that predicted confidence levels align with actual outcomes. Moreover, it holds significant reference value within the Blending ensemble strategy, as it safeguards the reliability of the probability outputs being integrated.

3.3.5 Cross Validation

To further enhance computational efficiency, this study implemented a Precomputed Folds mechanism integrated with the 10-fold cross-validation framework. Prior to the model training phase, the entire Zero-Leakage preprocessing pipeline—encompassing Target Encoding, Iterative Imputation, PCA-based dimensionality reduction, and feature pruning via Permutation Importance—was executed once for all folds. The result was subsequently cached in memory as a structured object (precomputed_10_folds).

Table 4. 10-Fold Cross-Validation Evaluation Metrics

Evaluation Indicators	Score
Accuracy	0.8792
F1-Score	0.7841
ROC-AUC	0.9215
Log Loss	0.3868

This architecture ensures that the time-consuming preprocessing steps do not need to be repeated when evaluating different hyperparameter configurations (Conservative, Moderate, and Aggressive). Consequently, this significantly reduces computational overhead and accelerates the experimental cycle .

4. RESULT

Our model demonstrated strong and stable performance using a 10-fold cross-validation blending strategy, the key metrics are as Table 4. Furthermore, the model achieved a highly competitive final score of 0.8826 on the Kaggle Public Leaderboard, and a final score of 0.8638 on the Private Leaderboard.

The cross-validation (CV) results and the Kaggle Public Leaderboard score are highly consistent. This consistency clearly indicates that our model generalizes well to unseen data without any severe overfitting issues. Overall, averaging the three blending configurations successfully improved the stacking model's stability and predictive performance.

5. WORK DIVISION

Name	Work description	Contribution
PO-LIEN LIAO 114423053	1. Abstract 2. Stacking Model Selection (Base Model & Final Predictor Model) 3. Result	25%
MING-JHE HO 114423069	1. Data Preprocessing & Imputation methods selection 2. Data Leakage Prevention Strategies	25%
XIN-HAN CHEN 114423014	1. Model Training and Validation	25%
CHENG-WEI LIAO 114423068	1. Feature Engineering and Feature Selection. 2. Using self_intro as a semantic feature.	25%